

CODEX AGENT KIT

Guia Definitivo dos Agentes

Mapa completo, pasta por pasta, arquivo por arquivo — o que cada agente faz, quando usá-lo, e como ser cirúrgico.

41

arquivos

14

pastas

11

agentes

4

validadores

Preparado por Claude • Maio de 2026 • ENT





Bem-vindo ao kit

O **Codex Agent Kit** é uma biblioteca de agentes especializados que vivem na pasta `.codex/` do seu projeto. Cada agente tem um prefixo semântico, uma especialidade clara e um lugar definido no pipeline.

Este guia foi montado **arquivo por arquivo**. Para cada um você verá: o que faz, no que ele é melhor, em quais situações usá-lo, suas regras-chave e como mencioná-lo no chat.

Use o **mapa visual** da próxima página para entender a estrutura, depois leia diretamente a seção do agente que você precisa.

Como ler cada ficha

■ O que faz	Resumo cirúrgico do papel do agente.
■ Onde brilha	Os pontos em que ele é insuperável.
■ Quando acionar	Situações concretas em que ele entra em campo.
■■ Regras-chave	Princípios não-negociáveis do agente.
■ Como mencionar	A tag que você usa no chat.
“ Frase-guia	A linha que define a postura do agente.



■■ Mapa da pasta .codex

Estrutura completa, organizada por prefixo. Cada agente vive em sua pasta com nome semântico.

Prefixo	Pasta	Área	Arquivos
C10	C10_Maestro	Maestro, orquestração e memória	7
C	C_Cetico	Revisão cética de planos	1
A	A_Architecture	Arquitetura cross-stack	1
BI	BI_Dashboards	Business Intelligence	4
D	D_Design	Design, UX e frontend	1
E	E_Environment	Environment e secrets	4
M	M_MobilePlaystore	Mobile RN + Expo + Play Store	6
O	O_Observability	Observabilidade e deploy	1
P	P_Performance	Performance e escalabilidade	1
PR	PR_PromptOps	Prompt engineering	2
Q	Q_Quality	QA e testes	1
S	S_Seguranca	Segurança	1
T	T_Templates	Templates reutilizáveis	5
V	V_Validation	Validadores e selo final	4
ROOT	.codex (raiz)	Catálogo, auditoria e guia	4

■ Pipeline recomendado

A ordem natural dos agentes, do briefing à entrega:

#	Agente	Função
1.	C10 Maestro	Entende o projeto, define fase e monta brief
2.	A Arquitetura	Valida desenho desacoplado, contratos e fronteiras
3.	V Impact Validator	Revisa plano e impacto cross-stack



4.	S Security Validator	Entra se tocar auth, PII, secrets, upload, headers, pagamentos
5.	P Performance Validator	Entra se tocar hot path, cache, queries, listas, filas, custo
6.	Executor (D / E / M / etc.)	Implementa com escopo controlado
7.	BI Dashboard Designer	Entra quando há métricas, relatórios e indicadores
8.	PR Prompt Refiner	Entra para lapidar pedidos em prompts cirúrgicos
9.	Q Test Engineer	Plano de testes e cenários de regressão
10.	V Final Validator	Selo de qualidade — revisa diff antes do merge
11.	C10 Documentador	Registra LOG, STATUS, ADRs e aprendizados



Fichas dos agentes

Daqui em diante, cada arquivo da pasta `.codex/` ganha sua ficha completa. Os agentes estão organizados por família (prefixo).

ROOT

Raiz

Catálogo, auditoria e guia da pasta `.codex`

AGENTS.md

`.codex` (raiz)

AGENTS.md — Catálogo Mestre

Ponto de entrada da pasta `.codex`

O que faz

Catálogo mestre da pasta. Define padrão de nomenclatura por prefixo, pipeline recomendado (11 etapas), regras globais de Vibe Coding, política de execução no Codex, padrão anti-alucinação e como mencionar cada agente no chat.

ONDE BRILHA

- Servir de mapa de toda a biblioteca `.codex`
- Listar prefixos, áreas e exemplos de cada agente
- Apontar pipeline: `C10 → A → V → S → P → executor → BI → PR → Q → V (final) → Documentador`
- Documentar as menções (`@C10`, `@A`, `@BI`, `@D`, `@E`, `@M`, `@P`, `@PR`, `@Q`, `@S`, `@V`)

QUANDO ACIONAR

- Está chegando na pasta `.codex` e precisa de visão geral
- Quer saber o que cada prefixo significa
- Precisa decidir qual agente acionar agora

REGRAS-CHAVE

- Nomes de pasta e arquivo sem acentos e sem espaços
- Para o Codex carregar instruções automaticamente, copiar `C10_Agent_ProjectRules.md` para `PROJECT_ROOT/AGENTS.md`
- Toda mudança protege a base oculta do iceberg, não só a ponta visual

Como mencionar:

Catálogo — leia primeiro

“Este arquivo é o ponto de entrada da pasta `.codex/`.”

AUDIT_AGENTES.md

`.codex` (raiz)



AUDIT_AGENTES.md — Relatório de auditoria

Auditoria interna dos agentes do kit

■ O que faz

Documento auxiliar com relatório/auditoria dos agentes. Pode conter histórico de evolução, gaps identificados e plano de melhoria.

■ ONDE BRILHA

- Servir de histórico de mudanças no kit
- Apontar gaps ou redundâncias

■ QUANDO ACIONAR

- Vai revisar a saúde do próprio kit
- Está adicionando agentes novos e quer ver o que existe

■ REGRAS-CHAVE

- Documento auxiliar, não impacta runtime

■ Como mencionar:

Documento auxiliar –
não se menciona

“Auditoria interna do kit.”

GUIA_COMPLETO_CODEX_AGENT_KIT.md

■ .codex (raiz)

Guia Completo (versão Markdown)

Guia textual do kit em Markdown

■ O que faz

Versão em Markdown deste mesmo guia, útil para leitura no editor ou geração de outras saídas.

■ ONDE BRILHA

- Leitura rápida no editor de texto
- Servir de fonte para outras conversões

■ QUANDO ACIONAR

- Quer ler sem abrir o PDF
- Vai versionar/diff o guia

■ REGRAS-CHAVE

- Mantém-se em sincronia com o PDF



■ **Como mencionar:**

Documento — não se menciona

“Versão Markdown do guia completo.”

GUIA_COMPLETO_CODEX_AGENT_KIT.pdf

■ .codex (raiz)

Guia Completo (versão PDF anterior)

Versão PDF anterior do guia

■ **O que faz**

Versão PDF anteriormente gerada do guia. Pode ser substituída pelo novo PDF gerado neste ciclo.

■ **ONDE BRILHA**

- Compartilhar guia em PDF

■ **QUANDO ACIONAR**

- Quer um PDF para imprimir ou enviar

■ ■ **REGRAS-CHAVE**

- Substituir/atualizar quando houver nova versão

■ **Como mencionar:**

Documento — não se menciona

“Versão PDF do guia completo (versão anterior).”

C10

C10 — Maestro & Memória

Orquestração, fluxo e governança do projeto

C10_CAMISA10.md

■ C10_Maestro

Camisa10 — O Maestro do Projeto

Orquestrador-chefe que comanda tudo, do primeiro commit ao deploy final

■ O que faz

É o agente sênior que enxerga o campo inteiro. Reconhece em qual das 6 fases o projeto está (Concepção, Fundação, Desenvolvimento, Integração, Hardening, Entrega), faz o onboarding inicial, distribui o jogo entre os outros agentes e nunca deixa nada perder o fio.

■ ONDE BRILHA

- Iniciar projetos novos (cria PROJECT.md, STATUS.md, LOG.md, DECISIONS.md, LEARNINGS.md)
- Coordenar o ciclo: ENTENDER → PLANEJAR → CÉTICO → AJUSTAR → EXECUTAR → VALIDAR → DOCUMENTAR
- Decidir qual sub-agente entra em campo a cada momento
- Manter a memória viva do projeto e o STATUS sempre atualizado

■ QUANDO ACIONAR

- Você está começando um projeto do zero
- Precisa retomar um projeto antigo e quer alguém com 'visão de campo'
- Quer garantir que a fase atual do projeto esteja clara
- Não sabe qual agente acionar — chame o Camisa10 antes

■ REGRAS-CHAVE

- Cético antes de implementar (sempre, sem exceção)
- Validador antes de fechar entrega
- Documentador fecha o ciclo (LOG, DECISIONS, LEARNINGS)
- STATUS.md é a verdade — nunca desatualizado por mais de 1 ciclo

■ Como mencionar:

@C10

“Você é rígido nas regras mas nunca arrogante na comunicação. Trata erros como dados, não como falhas morais.”

C10_DOCUMENTADOR.md

■ C10_Maestro

Documentador — Fechador de Ciclo

Transforma o que foi feito em conhecimento permanente



■ O que faz

Entra em campo depois que o Validador confirma uma entrega. Sua função é garantir que nada do ciclo se perca: registra LOG, DECISIONS (ADRs), LEARNINGS e atualiza STATUS.

■ ONDE BRILHA

- Escrever entradas cronológicas e objetivas no LOG.md
- Registrar Architecture Decision Records (ADRs) no DECISIONS.md
- Capturar erros, armadilhas, padrões e otimizações no LEARNINGS.md
- Atualizar fase, tarefas concluídas e bloqueios em STATUS.md

■ QUANDO ACIONAR

- Acabou de finalizar uma feature ou ciclo
- Houve uma decisão arquitetural que precisa de registro
- Apareceu um bug ou aprendizado que serve para o próximo projeto
- STATUS.md está desatualizado

■ REGRAS-CHAVE

- Brief incompleto = pergunta antes de documentar (vaga é pior que nenhuma)
- Sequência fixa: LOG → DECISIONS → LEARNINGS → STATUS → Relatório
- LOG é cronológico e imutável; LEARNINGS é matéria-prima para o guia de SDD

■ Como mencionar:

@C10 (Documentador)

“Documentação vaga é pior que nenhuma documentação.”

C10_Agent_ProjectRules.md

■ C10_Maestro

ProjectRules — Modelo de AGENTS.md do projeto

Template oficial para o AGENTS.md da raiz do projeto

■ O que faz

É o arquivo-modelo que deve ser copiado/adaptado para PROJECT_ROOT/AGENTS.md, lido automaticamente pelo Codex em cada sessão. Define identidade, stack, estrutura de pastas, convenções, comandos e fluxo de trabalho do projeto.

■ ONDE BRILHA

- Configurar o comportamento do agente Claude/Codex em um projeto específico
- Padronizar nomenclatura, estrutura de pastas e convenções
- Garantir que o agente leia PROJECT.md, STATUS.md e LOG.md antes de agir
- Apontar para o Camisa10 como maestro principal

■ QUANDO ACIONAR

- Está iniciando um projeto novo e precisa do AGENTS.md raiz
- Quer que o Codex tenha contexto automático em cada sessão
- Precisa documentar stack, convenções e fluxo de trabalho

■ REGRAS-CHAVE

- Personalizar com nome do projeto, stack real e estrutura concreta
- Lido automaticamente pelo Codex — manter sempre atualizado

■ Como mencionar:

Não se menciona — é
template para
AGENTS.md raiz

“Configura o comportamento do agente Claude/Codex neste projeto.”

C10_STATUS.md

■ C10_Maestro

STATUS.md — Modelo do estado vivo

Snapshot do agora: fase, tarefas em andamento, bloqueios

■ O que faz

Template do arquivo vivo que sempre reflete o estado presente do projeto. Histórico fica no LOG; aqui só vive o agora.

■ ONDE BRILHA

- Mostrar fase atual e última atualização
- Listar tarefas em andamento com responsáveis
- Indicar próximas tarefas e o que está concluído
- Sinalizar bloqueios que impedem avanço

■ QUANDO ACIONAR

- Precisa criar o STATUS.md inicial de um projeto
- Quer um snapshot rápido para retomada

■ REGRAS-CHAVE

- Não acumular histórico aqui — isso vai para o LOG
- Atualizar a cada ciclo (responsabilidade do Documentador)

■ Como mencionar:

Template — não se
menciona diretamente

“Sempre reflete o estado presente do projeto.”

C10_LOG.md

■ C10_Maestro

LOG.md — Modelo da cronologia imutável

Tudo que aconteceu, com data e status

■ O que faz

Template de cronologia completa do projeto. Cada entrada é imutável após criada. Análises ficam no LEARNINGS; aqui só fatos.

■ ONDE BRILHA

- Registrar entregas com [OK], [OK*], [PARCIAL] ou [REVERTIDO]
- Manter histórico cronológico para auditoria
- Servir como fonte da verdade temporal

■ QUANDO ACIONAR

- Vai criar o LOG inicial de um projeto
- Precisa registrar uma entrega, reversão ou marco

■ REGRAS-CHAVE

- Entradas mais recentes no topo
- Imutável após registrado — não reescreve história

■ Como mencionar:

Template — não se menciona

“Cronologia completa do projeto. Nada é apagado daqui.”

C10_DECISIONS.md

■ C10_Maestro

DECISIONS.md — Modelo do registro de ADRs

Architecture Decision Records com peso real

■ O que faz

Template no formato ADR. Apenas decisões com peso real — que mudariam o sistema se fossem diferentes. Estilo e detalhes de implementação não entram aqui.

■ ONDE BRILHA

- Registrar decisões com Aceita / Substituída / Revertida
- Manter histórico de raciocínio (por que foi escolhido X em vez de Y)
- Servir de referência para revisões futuras

■ QUANDO ACIONAR

- Vai tomar uma decisão arquitetural relevante
- Precisa documentar trade-offs

■ REGRAS-CHAVE

- Decisões substituídas/revertidas não são apagadas
- Sempre tem 'porquê' — sem justificativa não vira ADR

■ Como mencionar:

Template — não se
menciona

“Decisões têm porquê. Sem justificativa não é registrado.”

C10_LEARNINGS.md

■ C10_Maestro

LEARNINGS.md — Modelo do guia de aprendizados

Matéria-prima do guia de SDD

■ O que faz

Template para registrar Erros, Armadilhas, Padrões, Otimizações e Descobertas. Cada entrada é uma lição que o próximo projeto não vai precisar aprender do jeito difícil.

■ ONDE BRILHA

- Catalogar erros e como foram resolvidos
- Registrar padrões que funcionaram bem
- Capturar armadilhas evitadas pelo Cético
- Alimentar o guia de Specification-Driven Development

■ QUANDO ACIONAR

- Algo quebrou e foi consertado
- Uma abordagem deu muito certo e merece ser repetida
- Encontrou um padrão emergente

■ REGRAS-CHAVE

- Sem julgamento, sem culpados — só dados, causa raiz e o que fazer diferente
- Escrever como se fosse para alguém que nunca viu o projeto

■ Como mencionar:

Template — não se
menciona

“Cada entrada aqui é uma lição que o próximo projeto não vai precisar aprender do jeito difícil.”

C

C — Cético

Revisão cética antes de implementar

C_Agent_Cetico.md

■ C_Cetico

Cético Cirúrgico — Revisor de Planos

Confronta cada proposta contra o código real antes de implementar

■ O que faz

Não implementa nada. Sua função é revisar planos antes da execução, lendo código real, rastreando consumidores, separando fato de inferência e emitindo veredito proporcional à evidência. Sem evidência do repositório, não existe veredito técnico.

■ ONDE BRILHA

- Bloquear planos fracos, com risco alto ou duplicação grave
- Localizar superfície (arquivos, rotas, services, schemas, testes) antes de opinar
- Mapear fluxo atual e confrontar contra o plano
- Emitir um dos 4 vereditos: APROVADO / APROVADO COM RESSALVAS / QUESTIONAR / REJEITADO

■ QUANDO ACIONAR

- Antes de qualquer feature, refatoração, mudança de schema ou auth
- Antes de deploy, integração externa ou pagamento
- Antes de mudar performance ou segurança
- Sempre que o Camisa10 montar um plano

■ REGRAS-CHAVE

- Não aprovar sem ler arquivos diretamente afetados
- Não aprovar sem rastrear consumidores principais
- Não usar APROVADO em mudança de contrato público sem critério de compatibilidade
- Veredito padrão sem acesso ao repo: QUESTIONAR

■ Como mencionar:

@C

“Sem evidência do repositório, não existe veredito técnico.”

A

A — Arquitetura

Cross-stack, fronteiras e contratos

A_Agent_CrossStackArchitect.md

■ A_Architecture

Arquiteto Cross-Stack

Desenha sistemas desacoplados antes que o código vire definitivo

■ O que faz

Garante fronteiras claras entre frontend, backend, banco e workers. Define contratos versionados, valida integridade do banco, projeta idempotência, deduplicação, transações, paginação e observabilidade desde cedo.

■ ONDE BRILHA

- Mapear atores, casos de uso e fluxos críticos
- Definir endpoints, DTOs, schemas, eventos e webhooks
- Identificar hot paths e pontos de cache potencial
- Apontar próximo agente do pipeline (V/S/P/Q/O)

■ QUANDO ACIONAR

- Projeto novo está sendo concebido
- Frontend/backend/banco precisam ser separados
- Uma feature toca API, schema, auth, filas, webhooks ou jobs
- Existem componentes ou endpoints duplicados
- Há dúvida sobre onde uma regra de negócio deve viver

■ REGRAS-CHAVE

- UI nunca acessa banco direto
- Frontend nunca guarda secrets
- Backend é a autoridade de regra de negócio e autorização
- Banco protege integridade com constraints, índices e transações

■ Como mencionar:

@A

“Arquitetura boa reduz risco futuro sem criar complexidade teatral agora.”

BI

BI — Business Intelligence

Dashboards, métricas e relatórios

BI_Agent_DashboardDesigner.md

■ BI_Dashboards

BI Dashboard Designer

Cria salas de comando, não 'um monte de gráfico'

■ O que faz

Especialista em Business Intelligence. Cria dashboards úteis, bonitos, confiáveis, performáticos e seguros. Indicadores claros, hierarquia visual, filtros completos, métricas explicadas, drill-downs e dados confiáveis.

■ ONDE BRILHA

- Definir KPIs com fórmula, fonte, timezone e permissão
- Desenhar hierarquia visual: KPIs → tendência → composição → ranking → detalhe
- Especificar filtros de período (com calendário e presets) e filtros server-side
- Garantir paleta com significado, estados de UI e exportação segura

■ QUANDO ACIONAR

- Projeto envolve dashboard executivo, operacional, financeiro, comercial ou de produto
- Precisa modelar métricas e definir fonte da verdade
- Tem relatórios, KPIs, gráficos ou tabelas analíticas
- Há exportação CSV/PDF, snapshots ou relatórios recorrentes

■ REGRAS-CHAVE

- Frontend nunca acessa banco diretamente
- Toda métrica precisa de definição, fórmula e fonte
- Filtros server-side quando alteram dados
- RLS, tenant id e role checks em multi-tenant
- Cache com chave incluindo tenant + filtros + período + permissão

■ Como mencionar:

@BI

"Dashboard sem contexto, legenda e definição de métrica é decoração perigosa."

BI_Template_DASHBOARD_SPEC.md

■ BI_Dashboards

DASHBOARD_SPEC — Modelo de especificação

Briefing completo de um dashboard antes de codar

■ O que faz

Template para especificar usuário, decisão suportada, fontes de dados, filtros, KPIs, gráficos, permissões, performance e estados de UI.

■ ONDE BRILHA

- Descrever o usuário e a decisão que o dashboard apoia
- Mapear fontes de dados com dono, tipo e atualização
- Definir filtros de período, status, tenant, produto, etc.

■ QUANDO ACIONAR

- Vai começar a desenhar um dashboard novo
- Precisa alinhar requisitos com stakeholders antes de implementar

■ REGRAS-CHAVE

- Sempre ter campo de data principal e timezone explícito
- Listar presets: hoje, ontem, 7d, 30d, mês, trimestre, ano

■ Como mencionar:

Template — não se menciona

“Frequência de uso e decisão são parte da spec, não bonus.”

BI_Template_METRIC_DICTIONARY.md

■ BI_Dashboards

METRIC_DICTIONARY — Fonte da verdade das métricas

Cada métrica com fórmula, fonte e tratamento

■ O que faz

Template para catalogar todas as métricas do projeto com descrição, pergunta que responde, fórmula exata, fonte, campo de data, filtros padrão, unidade, permissão, tratamento de nulos e duplicidade.

■ ONDE BRILHA

- Eliminar 'métrica que cada um calcula de um jeito'
- Servir de contrato entre BI, produto e engenharia
- Documentar trade-offs e observações

■ QUANDO ACIONAR

- Tem mais de 3 métricas no produto
- Métricas são usadas em mais de um dashboard
- Há divergência entre relatórios

■ REGRAS-CHAVE

- Fórmula deve ser exata (não 'aproximadamente')
- Sempre indicar tratamento de nulos e duplicidade

■ Como mencionar:

Template — não se menciona

“Use este arquivo como fonte da verdade das métricas do projeto.”

BI_Template_DASHBOARD_QA.md

■ BI_Dashboards

DASHBOARD_QA — Checklist de qualidade

Validação de dados, visual, segurança e performance

■ O que faz

Template com checklist completo: valida dados (timezone, nulos, duplicidades), visual (KPIs, gráficos, cores, estados), segurança (permissão, tenant, PII, exportação) e performance.

■ ONDE BRILHA

- Garantir que métricas batem com query manual
- Verificar que tenant A não vê dados do tenant B
- Conferir estados de loading/vazio/erro/sem permissão

■ QUANDO ACIONAR

- Acabou de implementar ou alterar um dashboard
- Vai entregar para usuário final

■ REGRAS-CHAVE

- Exportação respeita filtros e permissões
- PII mascarada quando necessário

■ Como mencionar:

Template — não se menciona

“Garante que o dashboard ajuda a tomar uma decisão real.”

D

D — Design

UX e frontend visual com lógica preservada

D_Agent_Design.md

■ D_Design

Design + Codex Agent v2.0 Pro

Designer sênior + Engenheiro frontend + Guardiã funcional

■ O que faz

Híbrido designer/engenheiro com referências de Apple, Stripe, Linear e Vercel. Melhora interfaces sem quebrar lógica, fluxos, integrações, validações ou segurança. Funcionalidade sempre vence estética.

■ ONDE BRILHA

- Aplicar tokens de design (cores, tipografia, espaçamento, sombras)
- Refinar cards, botões, inputs e modais com padrão premium
- Garantir estados (default/hover/focus/disabled/loading/error/success)
- Manter performance (LCP < 2.5s, INP < 200ms, CLS < 0.1)

■ QUANDO ACIONAR

- Refatoração visual de componente ou tela
- Quer aplicar sistema de design consistente
- Precisa melhorar UX sem mexer em backend
- Quer unificar visual de uma feature antiga

■ REGRAS-CHAVE

- Funcionalidade sempre vence estética
- Nunca remove handler, validação, guard de rota ou regra de negócio
- Preserva acessibilidade e responsividade
- Comenta decisões não óbvias com 'DESIGN AGENT:'

■ Como mencionar:

@D

“Design não é apenas aparência. Design é funcionamento, clareza, confiança e continuidade.”

E

E — Environment

Variáveis, Vercel e secrets entre projetos

E_Agent_Environment.md

■ E_Environment

Vercel Environment Variables Manager

Audita variáveis dos projetos frontend e backend e suas referências cruzadas

■ O que faz

Especialista em variáveis de ambiente da Vercel para sistemas desacoplados. Gerencia DOIS projetos Vercel (Next.js + Express Serverless), audita ambos e valida consistência entre eles.

■ ONDE BRILHA

- Validar token Vercel antes de qualquer escrita
- Auditar via GET /env e gerar tabelas separadas frontend/backend
- Cross-check de NEXT_PUBLIC_API_URL × domínio backend e CORS_ORIGIN × domínio frontend
- Sinalizar secrets no frontend (DATABASE_URL, SERVICE_ROLE) e chaves Stripe live/test

■ QUANDO ACIONAR

- Vai configurar variáveis Vercel pela primeira vez
- Está com erro de CORS, JWT ou secret 'sumindo'
- Quer migrar de staging para produção sem chave Stripe trocada
- Auditoria de secrets antes do deploy

■ REGRAS-CHAVE

- Nunca expõe token completo (mostra ****xxxx)
- Confirmar com o usuário antes de POST/PATCH/DELETE
- Secrets só no backend; NEXT_PUBLIC_ é público
- Redeploy em ordem: backend antes do frontend

■ Como mencionar:

@E

“Uma variável errada em um lado quebra o outro.”

E_Reference_Nomenclature.md

■ E_Environment

Reference Nomenclature

Catálogo de variáveis por serviço (frontend e backend separados)

■ O que faz



Referência consultiva: regras gerais (UPPER_SNAKE_CASE, NEXT_PUBLIC_), e listas separadas de variáveis para Next.js (frontend) e Express (backend) com integrações Supabase, Stripe, Resend, OpenAI, AWS S3, etc.

■ ONDE BRILHA

- Mostrar quais variáveis cada projeto deve ter
- Diferenciar prefixo público (NEXT_PUBLIC_) de secret
- Servir como contrato de nomes para o stack

■ QUANDO ACIONAR

- Vai criar .env.example novo
- Tem dúvida sobre qual prefixo usar

■ REGRAS-CHAVE

- Qualquer secret com NEXT_PUBLIC_ é vulnerabilidade
- Backend não usa NEXT_PUBLIC_ em nada

■ Como mencionar:

Referência — não se menciona

“Frontend usa NEXT_PUBLIC_ para tudo que o browser precisa ler; backend não usa em nada.”

E_Reference_VercelAPI.md

■ E_Environment

Reference VercelAPI

Endpoints e exemplos da API Vercel

■ O que faz

Documentação dos endpoints da API Vercel para autenticação (GET /v2/user), listagem de projetos, GET/POST/PATCH/DELETE em /v9/projects/{id}/env e exemplos de resposta.

■ ONDE BRILHA

- Servir de cola técnica para chamadas HTTP
- Mostrar formato de payload e resposta
- Indicar headers obrigatórios

■ QUANDO ACIONAR

- Vai automatizar gestão de envs via script
- Precisa entender o que cada endpoint retorna

■ REGRAS-CHAVE

- Auth Header: Authorization: Bearer {VERCEL_TOKEN}
- API não retorna valores das variáveis (só metadados)

■ Como mencionar:

Referência – não se menciona

“Validar token antes de qualquer operação de escrita.”

E_Reference_CrossValidation.md

■ E_Environment

Reference CrossValidation

Checklist de consistência entre projetos Vercel

■ O que faz

Define todas as validações cruzadas: NEXT_PUBLIC_API_URL × domínio backend, CORS_ORIGIN × domínio frontend, JWT_SECRET único no backend, Supabase URL/keys, Stripe modo correto por environment.

■ ONDE BRILHA

- Detectar a causa #1 de quebra (URL da API errada)
- Detectar a causa #2 (CORS desalinhado)
- Sinalizar secrets duplicados indevidamente

■ QUANDO ACIONAR

- Frontend não consegue chamar backend
- Está com erro CORS em produção
- Vai promover de staging para prod

■ REGRAS-CHAVE

- Erros de cross-validation são a principal causa de falhas em sistemas desacoplados
- JWT_SECRET só no backend

■ Como mencionar:

Referência – não se menciona

“Erros de cross-validation são a principal causa de falhas em sistemas desacoplados.”

M

M — Mobile (RN + Expo)

Apps Android/iOS prontos para Play Store

M_Agent_MobilePlaystore.md

■ M_MobilePlaystore

Camisa 10 Mobile (RN + Expo + Play Store)

Apps profissionais em React Native + Expo prontos para Play Store

■ O que faz

Cria apps com frontend mobile e backend separados, prontos para uso offline, segurança, performance, EAS Build e Play Store. Stack default: RN + Expo + TypeScript + Expo Router + TanStack Query + Zod + SQLite + SecureStore.

■ ONDE BRILHA

- Onboarding completo (público, fluxos, offline, auth, integrações, políticas)
- Estrutura monorepo apps/mobile + api/
- Política de dados no device (token em SecureStore, dados em SQLite)
- Plano de release (interno → closed → open → produção)

■ QUANDO ACIONAR

- Vai criar app novo em React Native + Expo
- Precisa preparar app para Play Store
- App requer modo offline (leitura, fila ou offline-first)
- Auth mobile com biometria, social login ou JWT próprio

■ REGRAS-CHAVE

- Cria 6 documentos: PROJECT, ARCHITECTURE, OFFLINE, SECURITY, RELEASE, TEST_PLAN
- Senhas/API keys privadas: nunca no device
- Auth flow: refresh token + biometria opcional

■ Como mencionar:

@M

“Documentado antes de codar.”

M_Template_MOBILE_PROJECT.md

■ M_MobilePlaystore

MOBILE_PROJECT — Visão e stack do app

Template inicial: visão, plataformas, modo offline, fluxos

■ O que faz

Documento-mãe do app mobile: objetivo, público, plataformas (Android primeiro / Android+iOS), nível offline, stack, fluxos principais e dados no device.

■ ONDE BRILHA

- Capturar decisões iniciais antes do código
- Alinhar plataformas e prioridades
- Definir nível offline desejado

■ QUANDO ACIONAR

- Está iniciando um app novo
- Quer alinhar com cliente antes de codar

■ REGRAS-CHAVE

- Sempre indicar nível offline (nenhum / leitura / leitura+escrita / completo)
- Definir plataformas no início, não na metade

■ Como mencionar:

Template — não se menciona

“Define visão, stack, fluxos, plataformas e decisões.”

M_Template_MOBILE_OFFLINE.md

■ M_MobilePlaystore

MOBILE_OFFLINE — Estratégia offline

Fila local de mutações, conflitos, retry e backoff

■ O que faz

Template para estratégia offline: nível, entidades sincronizáveis, política de conflito (server wins / client wins / merge / revisão humana) e estrutura da fila local.

■ ONDE BRILHA

- Estruturar mutações com clientMutationId, payload, attempts, status
- Definir como resolver conflitos
- Documentar retry com backoff

■ QUANDO ACIONAR

- App tem qualquer nível offline
- Usuários têm internet ruim ou intermitente

■ REGRAS-CHAVE

- Toda mutação offline tem clientMutationId único
- Status flui: pending → syncing → synced (ou failed)

■ Como mencionar:

Template — não se menciona

“Estratégia offline, conflitos, fila local, retry e backoff.”

M_Template_MOBILE_SECURITY.md

■ M_MobilePlaystore

MOBILE_SECURITY — Política de segurança no device

Tokens, SecureStore, PII, permissões e logs

■ O que faz

Tabela de política de dados locais: o que pode ficar no device, em qual storage, com qual proteção. Token em SecureStore; PII só com justificativa; senhas e dados de pagamento nunca.

■ ONDE BRILHA

- Mapear permissões com motivo e fallback
- Política de logs (nunca logar token, senha, payload de pagamento)
- Hardening de auth e biometria

■ QUANDO ACIONAR

- App lida com dados pessoais ou financeiros
- Precisa passar revisão de segurança da Play Store

■ REGRAS-CHAVE

- Senhas/API keys privadas: nunca no device
- PII mínima necessária com controles

■ Como mencionar:

Template — não se menciona

“Tokens, SecureStore, PII, permissões e logs.”

M_Template_MOBILE_RELEASE.md

■ M_MobilePlaystore

MOBILE_RELEASE — Plano de release

EAS Build, versionamento, Play Store, rollback

■ O que faz

Template do plano de release com EAS Build + EAS Submit, versionamento, tracks da Play Store (interno, closed, open, produção), checklist e estratégia de rollback.



■ ONDE BRILHA

- Padronizar versionamento semântico + buildNumber
- Sequenciar tracks (interno → closed testing → open → produção)
- Checklist pré-publicação

■ QUANDO ACIONAR

- Vai publicar versão nova na Play Store
- Precisa coordenar release entre app e backend

■ REGRAS-CHAVE

- Sempre testar em closed testing antes de produção
- Ter plano de rollback antes de subir

■ Como mencionar:

Template — não se menciona

“EAS, versionamento, Play Store, checklist e rollback.”

M_Template_MOBILE_TEST_PLAN.md

■ M_MobilePlaystore

MOBILE_TEST_PLAN — Plano de testes

Unit, integration, device smoke, regressão

■ O que faz

Template para plano de testes do app: unit, integração com API, smoke em device/emulador e regressão. Casos de erro de rede, offline, permissão negada.

■ ONDE BRILHA

- Cobrir caminho feliz, erro de rede, offline e permissão negada
- Definir matriz de devices e versões Android
- Smoke test pós-build

■ QUANDO ACIONAR

- Antes de submeter para Play Store
- Antes de release com feature crítica

■ REGRAS-CHAVE

- Sempre testar offline → reconexão
- Testar permissão negada e fallback

■ Como mencionar:

Template — não se menciona

“Unit, integration, device smoke, regressão.”





O — Observability

Deploy, logs, métricas, alertas e rollback

O_Agent_DeployObservability.md

■ O_Observability

Deploy & Observability Agent

Garante que o sistema possa ser entendido, monitorado e recuperado

■ O que faz

Especialista em deploy, operação e observabilidade. Define ambientes separados, secrets por env, health checks, logs estruturados, métricas, alertas, traces, plano de rollback e smoke test pós-deploy.

■ ONDE BRILHA

- Logs estruturados com correlation/request id
- Métricas de latência, taxa de erro, throughput e saturação
- Alertas para auth, checkout, webhook, sync, fila e cron
- Plano de rollback e smoke test pós-deploy

■ QUANDO ACIONAR

- Antes de deploy de produção
- Ao criar backend novo, jobs, webhooks, pagamentos ou filas
- Há erro intermitente, timeout ou lentidão
- Vai integrar Sentry / OpenTelemetry / Prometheus

■ REGRAS-CHAVE

- Nunca logar token, senha, API key, documento completo ou payload de pagamento
- Erro com stack trace no observability, mas resposta segura ao usuário
- Veredito padrão sem evidências operacionais: QUESTIONAR

■ Como mencionar:

@O

“Logs devem responder: o que aconteceu, em qual request, com qual usuário, qual integração falhou, qual a duração, qual o resultado.”



P

P — Performance

Validação cética de otimizações

P_Agent_PerformanceValidator.toml

■ P_Performance

Performance Validator (Vector / Gauge / Pulse / Tempo)

Cético contra otimização sem medição

■ O que faz

Sub-agente em modo read-only. Não implementa — valida planos e diffs que prometem melhorar performance ou que podem regredir. Performance sem medição vira superstição; otimização que quebra semântica é bug.

■ ONDE BRILHA

- Identificar hot paths e baseline
- Avaliar cache, fila, índice, debounce, memoização (compensam o custo?)
- Detectar N+1, full scan, paginação ausente, transação longa
- Validar bundle, re-render, imagem, lista grande, CLS/INP/LCP

■ QUANDO ACIONAR

- Plano toca cache, queries, índices, hot paths
- Lista grande, imagem, bundle, renderização
- Concorrência, filas, jobs, polling
- Custo de API, memória, CPU, cold start

■ REGRAS-CHAVE

- Cache por usuário precisa incluir usuário/tenant/role na chave
- Toda otimização em lista grande exige paginação ou virtualização
- Vereditos: APROVADO / APROVADO COM RESSALVA / QUESTIONAR / REJEITADO

■ Como mencionar:

@P

“Performance sem medição vira superstição.”

PR

PR — Prompt Ops

Briefings cirúrgicos e prompts assertivos

PR_Agent_PromptRefiner.md

■ PR_PromptOps

Prompt Refiner Cirúrgico

Transforma ideias soltas em prompts precisos, seguros e acionáveis

■ O que faz

Entra em ação quando o usuário pede 'gera um prompt', 'melhora esse prompt', 'cria um briefing'. Prompt bom não é bonito: é verificável. Reduz ambiguidade, aponta contexto, limita escopo, protege o existente e define como validar.

■ ONDE BRILHA

- Gerar 4 tipos de prompt: Investigação, Plano, Implementação Segura, Validação
- Forçar leitura de código real antes de editar
- Incluir regras de segurança, performance e anti-duplicação
- Indicar agentes do pipeline que devem entrar (C10/C/A/V/S/P/Q/D/BI/M/O)

■ QUANDO ACIONAR

- Vai pedir algo grande para o Codex/Claude/ChatGPT
- Quer transformar uma ideia vaga em pedido cirúrgico
- Precisa de briefing antes de uma feature complexa

■ REGRAS-CHAVE

- Nunca gera prompt que peça para expor secrets, ignorar auth ou conectar front no banco
- Toda implementação prompted exige validações (build, lint, typecheck, testes)
- Manda verificar duplicação antes de criar nova solução

■ Como mencionar:

@PR

"Prompt bom não é bonito: é verificável."

PR_Template_PROMPT_BRIEF.md

■ PR_PromptOps

PROMPT_BRIEF — Modelo de briefing

Esqueleto do prompt cirúrgico

■ O que faz

Template com seções para Objetivo, Contexto (projeto, stack, fase), Evidências (arquivos verificados), Escopo (pode/não pode mexer), Riscos (segurança, performance), Plano e Validações.

■ ONDE BRILHA

- Forçar pensamento estruturado antes de pedir
- Servir como input para o Cético e Validador
- Reduzir ambiguidade do pedido

■ QUANDO ACIONAR

- Antes de pedir uma feature complexa
- Vai pedir para outro agente implementar

■ REGRAS-CHAVE

- Sempre listar 'pode mexer' e 'não pode mexer'
- Riscos explícitos (segurança e performance)

■ Como mencionar:

Template — não se
menciona

“Esqueleto do prompt cirúrgico.”



Q — Quality

QA, testes e regressão

Q_Agent_TestEngineer.md

■ Q_Quality

Test Engineer (QA Strategist)

Transforma risco em verificação objetiva

■ O que faz

Agente de QA e estratégia de testes. Define matriz por risco, escreve critérios de aceite testáveis, sugere unit/integration/contrato/e2e/smoke e identifica regressão provável.

■ ONDE BRILHA

- Garantir caminhos de erro, loading, vazio, permissão negada e timeout
- Casos: caminho feliz, input inválido, não autenticado, sem permissão, timeout, race condition
- Plano de testes para mobile offline/sync
- Regressão de contrato API

■ QUANDO ACIONAR

- Antes de fechar feature relevante
- Final_validator apontou falta de cobertura
- Fluxo crítico: auth, pagamento, upload, checkout, dashboard
- Antes de release mobile ou deploy de produção

■ REGRAS-CHAVE

- Verifica comportamento observável, não detalhes irrelevantes
- 12 itens obrigatórios de checklist (caminho feliz a contrato API)

■ Como mencionar:

@Q

“Transforma risco em verificação objetiva.”

S

S — Segurança

Validação cética de correções de segurança

S_Agent_SecurityValidator.toml

■ S_Seguranca

Security Validator (Cerberus / Aegis / Sentinel / Vanguard)

Cético de correções de segurança

■ O que faz

Sub-agente em modo read-only. NÃO encontra vulnerabilidades do zero, NÃO implementa patches — apenas valida correções de segurança propostas. Não fazer nada é melhor que fazer errado.

■ ONDE BRILHA

- Recusar fixes desnecessários ou que aumentam superfície de ataque
- Identificar quando já existe mitigação em outra camada (WAF, gateway, middleware)
- Distinguir correção real de mascarar problema
- Validar contexto: o problema é real neste projeto?

■ QUANDO ACIONAR

- Auditoria SAST/DAST/SCA gerou lista de fixes
- Outro agente sugeriu correções de segurança
- Existe AUDITORIA.md com itens a validar
- Plano toca auth, PII, upload, render de input externo

■ REGRAS-CHAVE

- Sem AUDITORIA.md, PARE e pergunte
- 4 eixos de validação por item
- Nunca aprova quando a correção introduz mais superfície que remove

■ Como mencionar:

@S

“Não fazer nada é melhor que fazer errado.”



T

T — Templates

Templates universais reutilizáveis

T_Template_PROJECT.md

■ T_Templates

PROJECT.md — Modelo do documento-mãe

Visão, arquitetura, fronteiras e fluxos críticos

■ O que faz

Template universal de PROJECT.md: visão, arquitetura (frontend/backend/banco/hospedagem/integrações), fronteiras claras e fluxos críticos.

■ ONDE BRILHA

- Capturar decisões iniciais do projeto
- Servir de leitura obrigatória para qualquer agente
- Definir o que é cada camada

■ QUANDO ACIONAR

- Está começando um projeto
- Camisa10 vai fazer onboarding

■ REGRAS-CHAVE

- Lista fluxos críticos (auth, cadastro, fluxo principal, pagamento)
- Define fronteiras explícitas

■ Como mencionar:

Template — não se menciona

“Documento-mãe lido por todos os agentes.”

T_Template_STATUS.md

■ T_Templates

STATUS.md — Modelo genérico

Estado vivo simplificado

■ O que faz

Template genérico de STATUS.md, alternativo ao C10_STATUS, para projetos que não vão usar todo o ritual do Camisa10.

■ ONDE BRILHA

- Snapshot rápido do agora
- Tarefas em andamento e bloqueios

■ QUANDO ACIONAR

- Projeto pequeno sem todo o pipeline
- Quer só status, sem todo o ritual

■ REGRAS-CHAVE

- Atualiza a cada ciclo
- Não vira histórico

■ Como mencionar:

Template — não se
menciona

“Snapshot do agora.”

T_Template_LOG.md

■ T_Templates

LOG.md — Modelo genérico

Cronologia simplificada

■ O que faz

Template genérico de LOG, alternativo ao C10_LOG, mais leve para projetos que dispensam o pipeline completo.

■ ONDE BRILHA

- Registrar entregas em ordem cronológica
- Marcadores OK / OK* / PARCIAL / REVERTIDO

■ QUANDO ACIONAR

- Projeto pequeno
- Não usa todo o ritual do Camisa10

■ REGRAS-CHAVE

- Imutável após registrado
- Mais recente no topo

■ Como mencionar:

Template — não se
menciona

“Cronologia simplificada.”

T_Template_DECISIONS.md

■ T_Templates

DECISIONS.md — Modelo genérico*ADRs sem todo o ritual***■ O que faz**

Template genérico de DECISIONS, alternativo ao C10_DECISIONS para projetos que precisam só de ADRs simples.

■ ONDE BRILHA

- Registrar ADRs com status Aceita/Substituída/Revertida
- Manter histórico de raciocínio

■ QUANDO ACIONAR

- Projeto pequeno
- Quer ADRs sem o restante do pipeline

■ REGRAS-CHAVE

- Decisões substituídas não são apagadas
- Tem porquê obrigatório

■ Como mencionar:

Template — não se menciona

“Apenas decisões com peso real.”

T_Template_LEARNINGS.md

■ T_Templates

LEARNINGS.md — Modelo genérico*Aprendizados simplificados***■ O que faz**

Template genérico de LEARNINGS, alternativo ao C10_LEARNINGS para projetos pequenos que querem capturar lições.

■ ONDE BRILHA

- Catalogar Erros, Armadilhas, Padrões, Otimizações, Descobertas
- Alimentar guia de SDD

■ QUANDO ACIONAR

- Projeto pequeno
- Quer só registro de lições

■ ■ REGRAS-CHAVE

- Sem julgamento, só dados
- Causa raiz e o que fazer diferente

■ Como mencionar:

Template – não se
menciona

*“Cada entrada é uma lição que o próximo projeto não vai
precisar aprender do jeito difícil.”*

V

V — Validation

Trio de validadores + selo final

V_Agent_ImpactValidator.toml

■ V_Validation

Impact Validator (Compass / Atlas / Lighthouse / Mosaic)

Mapeamento cross-stack do plano em 9 dimensões

■ O que faz

Sub-agente em modo read-only. Valida planos antes de implementar, mapeando impacto em 9 dimensões (contrato, interface, banco, frontend, jobs, integrações, infra, testes, docs) e 5 eixos (valor, impacto, duplicação, gambiarra, delegação). É o orquestrador do trio de validadores.

■ ONDE BRILHA

- Detectar over-engineering, gambiarra disfarçada, mudança duplicada
- Mapear o que o plano deixou de fora
- Decidir quando delegar para security_validator ou performance_validator
- Apontar ordem de implementação e estratégia de rollback

■ QUANDO ACIONAR

- Antes de aplicar qualquer plano vindo de outro agente, RFC ou arquivo .md
- Pediu 'isso vai quebrar?', 'isso afeta algo?', 'vale a pena implementar?'
- Plano colado direto no chat

■ REGRAS-CHAVE

- Hipótese nula: 'este plano precisa de ajustes ou não deve ser feito'
- Permissão para votar 'não fazer nada' em 5 cenários
- Vereditos: APROVADO / APROVADO COM RESSALVA / REJEITADO / QUESTIONAR
- Não decidir sozinho sobre segurança ou performance — delegar

■ Como mencionar:

@V (impact)

“Toda mudança tem custo. Minha função é mapear o impacto antes de escrever a primeira linha.”

V_Agent_ImpactGuard.md

■ V_Validation

Impact Guard — Snippet do AGENTS.md

Trecho colável que ativa o impact_validator + orquestração com S e P

■ O que faz

Documentação para colar no AGENTS.md raiz: explica como spawn o impact_validator e como ele orquestra security_validator e performance_validator quando detecta superfícies sensíveis.

■ ONDE BRILHA

- Ativar o trio de validadores no projeto
- Documentar quando cada um é acionado
- Padronizar o vocabulário (5 eixos, 9 dimensões)

■ QUANDO ACIONAR

- Vai configurar o pipeline no AGENTS.md raiz
- Quer que outros agentes saibam invocar os validadores

■ REGRAS-CHAVE

- Spawn antes de qualquer implementação
- Modo read-only sempre

■ Como mencionar:

Snippet — colar no AGENTS.md

“Validação cruzada — orquestração dos três agentes.”

V_Agent_FinalValidator.toml

■ V_Validation

Final Validator (Vouch / Hallmark / Gatekeeper / Sigil)

Selo de qualidade final pós-implementação

■ O que faz

Último portão antes do merge/deploy. Sub-agente read-only que revisa DIFFS (não arquivos inteiros) contra o plano de referência, detectando scope creep, bugs sutis, regressões, gambiarras, duplicações e quebras silenciosas.

■ ONDE BRILHA

- Detectar scope drift comparando plano vs entregue (DONE/PARTIAL/NOT DONE/OUT OF SCOPE)
- 7 eixos: corretude, regressão, duplicação, gambiarra, segurança, performance, testabilidade
- Identificar bugs típicos de IA (off-by-one, null handling, async errado, encoding, race condition)
- Delegar de volta para security_validator ou performance_validator se detectar regressão

■ QUANDO ACIONAR

- Acabou de implementar e quer revisão antes do merge
- PR está pronto para deploy
- Pediu 'tem bug aí?', 'fugiu do escopo?', 'pronto para mergear?'

■ ■ REGRAS-CHAVE

- Lê CADA linha do diff, não amostra
- Reporta apenas o que tem evidência concreta
- Não reescreve código — aponta o problema
- Vereditos: APROVADO / APROVADO COM RESSALVA / REPROVADO / QUESTIONAR

■ Como mencionar:

@V (final)

“Código que parece certo e não funciona é pior que código que não foi escrito.”

V_Agent_QualitySeal.md

■ V_Validation

Quality Seal — Snippet do AGENTS.md

Trecho colável do final_validator + diagrama do quarteto

■ O que faz

Documentação para colar no AGENTS.md raiz, explicando o final_validator e mostrando o diagrama do quarteto (impact + security + performance + final). Nada vai para produção sem passar pelo selo final.

■ ONDE BRILHA

- Ativar o final_validator no projeto
- Mostrar o pipeline completo (antes da impl + impl + depois da impl)
- Definir regras de bloqueio (REJEITADO/REPROVADO bloqueiam)

■ QUANDO ACIONAR

- Vai configurar o AGENTS.md raiz
- Quer documentar o pipeline para outros devs/agentes

■ ■ REGRAS-CHAVE

- Sempre acionado antes do merge/deploy
- Pode delegar de volta para security ou performance se detectar regressão

■ Como mencionar:

Snippet — colar no AGENTS.md

“Nada vai para produção sem passar pelo selo final.”

■ Receitas rápidas

Combinações de agentes prontas para situações reais.

Começar projeto novo

- 01 Acione @C10 e responda às 7 perguntas de onboarding
- 02 Camisa10 cria PROJECT.md, STATUS.md, LOG.md, DECISIONS.md, LEARNINGS.md
- 03 Adapte C10_Agent_ProjectRules.md como AGENTS.md raiz
- 04 Camisa10 chama @A para validar arquitetura inicial

Implementar feature em projeto existente

- 01 Camisa10 monta brief com fase, arquivos, dependências e plano
- 02 @C (Cético) revisa contra código real e emite veredito
- 03 @V (impact_validator) mapeia 9 dimensões cross-stack
- 04 @S e @P entram se tocar superfícies sensíveis
- 05 Worker implementa o plano aprovado
- 06 @V (final_validator) revisa o diff antes do merge
- 07 @C10 (Documentador) fecha o ciclo (LOG, ADR, LEARNINGS)

Refatorar visual sem quebrar lógica

- 01 @D lê o componente inteiro e mapeia handlers, hooks e contratos
- 02 @D aplica tokens de design preservando funcionalidade
- 03 @Q valida estados (loading, error, empty, sem permissão)
- 04 @V (final) revisa diff e libera para merge

Configurar Vercel pela primeira vez

- 01 @E coleta token e PROJECT_IDs dos dois projetos
- 02 @E audita frontend e backend separadamente
- 03 @E cross-checa NEXT_PUBLIC_API_URL × CORS_ORIGIN × JWT_SECRET
- 04 @E aplica mudanças com confirmação única
- 05 Redeploy: backend primeiro, depois frontend

Lançar app na Play Store

- 01 @M conduz onboarding mobile completo
- 02 @M cria 6 docs: PROJECT, ARCHITECTURE, OFFLINE, SECURITY, RELEASE, TEST_PLAN
- 03 @Q define matriz de testes (offline, permissão negada, retry)
- 04 @O define logs, métricas e alertas
- 05 EAS Build → closed testing → open testing → produção

Construir dashboard analítico

- 01 @BI faz onboarding (usuário, decisão, fontes, permissões)
- 02 @BI preenche METRIC_DICTIONARY com fórmula, fonte e timezone



- 03 @A garante backend agregando dados (não front no banco)
- 04 @P valida queries (índices, paginação, cache por tenant)
- 05 @BI roda DASHBOARD_QA antes de entregar



Cada arquivo lido. Cada papel mapeado. Pipeline pronto para usar.

Use este guia como referência viva.
Cada vez que o kit evoluir, o guia também deve evoluir.

41 arquivos



14 pastas



11 agentes ativos



Pipeline cirúrgico